

BetTrace

A Retrieval-Augmented Generation System for Querying Betting Records

Mojtaba Eshghie

January 28, 2026

Outline

- 1 Introduction
- 2 System Architecture
- 3 Query Pipeline
- 4 Key Design Decisions
- 5 Live Examples
- 6 Results
- 7 Conclusion

Problem Statement

Business Need:

- Query betting records in natural language
- Mix of structured and unstructured queries
- Need accurate numerical computations
- Evidence-based answers with citations

Technical Challenges:

- LLMs cannot reliably do math
- Data contains typos (“Rapters” vs “Raptors”)
- Queries range from lookups to analysis
- Must provide grounded responses

Key Insight

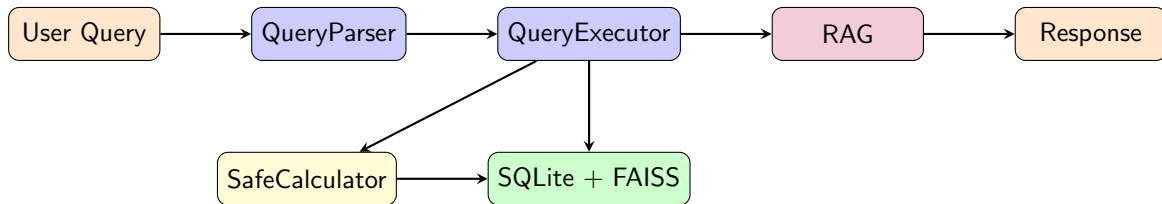
Traditional RAG fails when queries require computation or when data contains spelling variations.

Field	Description
bet_id	Unique identifier (e.g., B0042)
customer_id	Customer reference (e.g., C068)
sport	football, tennis, basketball
event_name	"Bulls vs Raptors", "Lakers vs Celtics"
status	SETTLED, PENDING, VOID, REJECTED
incident_tag	NONE, LATENCY_SPIKE, FEED_OUTAGE
price_delay_ms	Latency in milliseconds

Data Quality Issue

Event names contain typos: "Raptors", "Marsielle", "Lakers vs Celtics"

High-Level Architecture



Design Principle

Separate query understanding, data retrieval, computation, and response generation.

Component Responsibilities

Component	Responsibility
QueryParser	Extract entities, filters, aggregations from NL
QueryExecutor	Route queries to optimal execution strategy
Retriever	Hybrid search (SQL + semantic vector search)
SafeCalculator	All arithmetic via SQL
RAGAssistant	Context building, LLM calls, citations

Critical Decision

SafeCalculator performs ALL arithmetic via SQL. LLM never counts or averages.

Stage 1: Query Parsing

Example: “Why was bet B0004 rejected?”

```
DEBUG: Extracting bet IDs from query  
DEBUG: Extracting filters from query
```

```
ParsedQuery(  
    type=entity_lookup,  
    bet_ids=['B0004'],  
    filters={'status': 'REJECTED'},  
    semantic=['why']  
)
```

Key Insight

Parser extracts structured info (B0004, REJECTED) while preserving semantic intent (“why”).

Stage 2: Query Routing

Query types and their execution paths:

- **ENTITY_LOOKUP**: Has bet_id or customer_id → Direct SQL
- **AGGREGATE**: Has count/sum/avg + filters → SafeCalculator
- **TOP_N**: Has “top 5” + sort column → SQL ORDER BY
- **HYBRID**: Has filters + semantic terms → SQL + Vector rerank
- **SEMANTIC**: Team/player search → Phonetic + Vector search

Stage 3: SafeCalculator

Problem

LLMs hallucinate arithmetic. “Count these 7 items” might return 6 or 8.

Solution: All computation via SQL:

```
SELECT
    COUNT(*) as total_bets,      -- 4
    SUM(stake_gbp) as total,     -- 95.00
    AVG(stake_gbp) as avg_stake -- 23.75
FROM bets
WHERE status='VOID' AND incident_tag='FEED_OUTAGE'
```

LLM receives: “Total Bets: 4, Total Stake: 95.00 – USE THESE EXACT VALUES”

Design 1: Team-Level Embeddings

Problem

“Bulls vs Raptors” dilutes team signal. Query “Raptors” won’t match.

Solution: Separate embeddings for each team

“Bulls vs Raptors” $\rightarrow$ parse $\rightarrow$ (“Bulls”, “Raptors”)
 $\downarrow$
team1_embedding, team2_embedding

Search Strategy

Compare query against team1 OR team2, take MAX similarity.

Design 2: Phonetic Normalization

Problem

Embeddings don't see "Raptors" and "Rapters" as similar enough.

Solution: Normalize to phonetic codes before embedding

Original	→	Phonetic
"Raptors"	→	RPTRS
"Rapters"	→	RPTRS ✓ Same!
"Lakers"	→	LKRS
"Lakkers"	→	LKRS ✓ Same!
"Thunder"	→	THNDR
"Thunderr"	→	THNDR ✓ Same!

Phonetic Search in Action

Query: “All bets involving Thundr” (missing 'e')

```
DEBUG: Phonetic normalizing: 'Thundr'  
DEBUG: Embedding single text: 'THNDR'  
DEBUG: semantic_search_teams called
```

Result: Found 6 bets for “Clippers vs Thunderr”

- Query: “Thundr” (missing 'e')
- Data: “Thunderr” (extra 'r')
- Both normalize to THNDR → **Match!**

Example 1: Entity Lookup

Query: “Why was bet B0004 rejected?”

Parsed: type=entity_lookup, bet_ids=['B0004'], filters={status: REJECTED}

Answer:

```
The bet was rejected because the market was  
suspended (Incident: MARKET_SUSPENDED).
```

```
The delay was 635ms (elevated vs ~200ms normal  
but below 1000ms threshold).
```

```
Evidence: [B0004]
```

Example 2: Aggregate with Analysis

Query: “How many bets were voided due to FEED_OUTAGE? List them.”

Computed Facts: Total: 4, Stake: 95.00, Avg Delay: 960ms

Answer:

There were 4 bets voided due to FEED_OUTAGE:

- B0028: Zverev vs Tsitsipas, Delay: 495ms
- B0050: Man Utd vs Newcastle, Delay: 1073ms (high)
- B0066: Warriors vs Suns, Delay: 1078ms (high)
- B0094: Medvedev vs Rune, Delay: 1194ms (highest)

Key insight: FEED_OUTAGE affected multiple sports.

Evidence: [B0028, B0050, B0066, B0094]

Example 3: Customer Summary

Query: "For customer C068, summarize all bets and highlight incidents."

Computed: 4 bets, 115.00 total, all SETTLED, no incidents

Answer:

- B0003: PSG vs Marseille, SETTLED, NONE, 290ms
- B0018: Brighton vs Wolves, SETTLED, NONE, 222ms
- B0019: Man City vs Brentford, SETTLED, NONE, 121ms
- B0055: Raducanu vs Pegulla, SETTLED, NONE, 312ms

Key insights:

- All 4 bets SETTLED with no incidents
- B0055 has maximum delay (312ms)

Evidence: [B0003, B0018, B0019, B0055]

82 Tests Passing

Parser Tests:

- Bet/Customer ID extraction
- Status/incident filters
- Aggregation detection
- ID normalization

Retrieval Tests:

- Phonetic normalization
- Team search
- Hybrid search
- Citation extraction

Key Contributions:

- ① **Hybrid Query Pipeline:** SQL + vector search routing
- ② **SafeCalculator:** No LLM arithmetic errors
- ③ **Phonetic Normalization:** Handles data typos
- ④ **Team-Level Embeddings:** Precise team matching
- ⑤ **Analysis Intent Detection:** Adapts response detail
- ⑥ **Citation Enforcement:** Grounded answers

Thank You

Questions?